ЛАБОРАТОРНАЯ РАБОТА № 3.
Трансформация и визуализация данных

Цель работы – получить навыки трансформации и визуализации данных с использованием средств языка программирования R.

3.1. Библиотека pivottabler

Пакет pivottabler предназначен для создания и форматирования сводных таблиц. Все вычисления для сводных таблиц происходят внутри R. Сводные таблицы отображаются в htmlwidgets, Latex или обычный текст. Результаты обработки данных можно преобразовать в стандартные структуры данных R с использованием функции basictabler, а также экспортировать в Html, Excel, Word, PowerPoint.и другие форматы с использованием функции flextabler. Более подробная информацию представлена на сайте http://www.pivottabler.org.uk/articles .

Последнюю версию пакета pivottabler можно установить командой:

```
install.packages("pivottabler")
```

Рассмотрим пример отображении сводных таблиц в текстовом виде.

Простой пример создания сводной таблицы, обобщающей типы поездов, эксплуатируемых различными железнодорожными компаниями.

```
library(pivottabler)
# arguments:  qpvt(dataFrame, rows, columns, calculations, ...)
qpvt(bhmtrains, "TOC", "TrainCategory", "n()") # TOC = Train Operating Company
                     Express Passenger  Ordinary Passenger  Total
Arriva Trains Wales               3079                830   3909
CrossCountry                     22865                 63  22928
London Midland                   14487              33792  48279
Virgin Trains                     8594                      8594
Total                            49025              34685  83710
```

В pivottabler имеются дополнительные возможности, которые недоступны в функции qpvt. Команды для вывода той же сводной таблицы, что и выше, представлены далее.

```
pt <- PivotTable$new()
pt$addData(bhmtrains) # bhmtrains is a data frame with columns TrainCategory, TOC, etc.
pt$addColumnDataGroups("TrainCategory") # e.g. Express Passenger
pt$addRowDataGroups("TOC") # TOC = Train Operating Company e.g. Arriva Trains Wales
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$evaluatePivot()
pt
```

К заголовкам строк или столбцов сводной таблицы можно добавлять несколько уровней, например, рассматривая комбинации TOC и PowerType.

```
qpvt(bhmtrains, c("TOC", "PowerType"), "TrainCategory", "n()")
pt <- PivotTable$new()
pt$addData(bhmtrains)
pt$addColumnDataGroups("TrainCategory")
pt$addRowDataGroups("TOC")
pt$addRowDataGroups("PowerType") # D/EMU = Diesel/Electric Multiple Unit, HST=High
Speed Train
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$evaluatePivot()
pt
```

```
                          Express Passenger  Ordinary Passenger  Total
Arriva Trains Wales  DMU                3079                 830   3909
                     Total              3079                 830   3909
CrossCountry         DMU               22133                  63  22196
                     HST                 732                      732
                     Total             22865                  63  22928
London Midland       DMU                5638                5591  11229
                     EMU                8849               28201  37050
                     Total             14487               33792  48279
Virgin Trains        DMU                2137                      2137
                     EMU                6457                      6457
                     Total              8594                      8594
Total                                  49025               34685  83710
```

Отображение в HTML формате представленных выше сводных таблиц производится с помощью функции pt$renderPivot().

Первая таблица представлена на рисунке 3.1.

```
qhpvt(bhmtrains, "TOC", "TrainCategory", "n()")
pt <- PivotTable$new()
pt$addData(bhmtrains)
pt$addColumnDataGroups("TrainCategory")
pt$addRowDataGroups("TOC")
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$renderPivot()
```

	Express Passenger	Ordinary Passenger	Total
Arriva Trains Wales	3079	830	3909
CrossCountry	22865	63	22928
London Midland	14487	33792	48279
Virgin Trains	8594		8594
Total	49025	34685	83710

Рисунок 3.1 – Простая сводная таблица

Более подробная таблица представлена на рисунке 3.2.

```
qhpvt(bhmtrains, c("TOC", "PowerType"), "TrainCategory", "n()")
pt <- PivotTable$new()
pt$addData(bhmtrains) # bhmtrains is a data frame with columns TrainCategory, TOC, etc.
pt$addColumnDataGroups("TrainCategory") # e.g. Express Passenger
pt$addRowDataGroups("TOC") # TOC = Train Operating Company e.g. Arriva Trains Wales
pt$addRowDataGroups("PowerType") # D/EMU = Diesel/Electric Multiple Unit, HST=High
Speed Train
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$renderPivot()
```

		Express Passenger	Ordinary Passenger	Total
Arriva Trains Wales	DMU	3079	830	3909
	Total	3079	830	3909
CrossCountry	DMU	22133	63	22196
	HST	732		732
	Total	22865	63	22928
London Midland	DMU	5638	5591	11229
	EMU	8849	28201	37050
	Total	14487	33792	48279
Virgin Trains	DMU	2137		2137
	EMU	6457		6457
	Total	8594		8594
Total		49025	34685	83710

Рисунок 3.2 – Подробная сводная таблица

Макет структуры — это альтернативный способ визуализации групп строк, например, для той же сводной таблицы, что и выше представлен на рисунке 3.3.

```
pt <- PivotTable$new()
pt$addData(bhmtrains)
pt$addColumnDataGroups("TrainCategory")
pt$addRowDataGroups("TOC",
                    outlineBefore=list(isEmpty=FALSE,
groupStyleDeclarations=list(color="blue")),
                    outlineTotal=list(isEmpty=FALSE,
groupStyleDeclarations=list(color="blue")))
pt$addRowDataGroups("PowerType", addTotal=FALSE)
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$renderPivot()
```

	Express Passenger	Ordinary Passenger	Total
Arriva Trains Wales	3079	830	3909
DMU	3079	830	3909
CrossCountry	22865	63	22928
DMU	22133	63	22196
HST	732		732
London Midland	14487	33792	48279
DMU	5638	5591	11229
EMU	8849	28201	37050
Virgin Trains	8594		8594
DMU	2137		2137
EMU	6457		6457
Total	49025	34685	83710

Рисунок 3.3 – Форматированная сводная таблица

Макет структуры также можно использовать для построения сводной таблицы с переменной глубиной иерархии строк, например, простого баланса (рисунок 3.4).

	Budget	Actual	Variance
Net entrepreneurial income	**2824.4**	**2905.2**	**80.8**
Net operating surplus	**3295.5**	**3412.6**	**117.1**
Factor income	**4540.5**	**4641.9**	**101.4**
Net value added	**1765.3**	**1733.1**	**-32.2**
Gross value added	**3852**	**3718.1**	**-133.9**
Production of the agricultural industry	**10151**	**10783.4**	**632.4**
Crop production	4210.9	4373.7	162.8
Livestock production	4857.7	5307.6	449.9
Production of agricultural services	676.6	693.9	17.3
Other production	405.8	408.2	2.4
Intermediate services	-6299	-7065.3	-766.3
Depreciation	-2086.7	-1985	101.7
Taxes and subsidies	**2775.2**	**2908.8**	**133.6**
Other taxes on production	-145.4	-154.2	-8.8
Other subsidies (non-product specific)	2920.6	3063	142.4
Compensation of employees	-1245	-1229.3	15.7
Interests and rents	**-471.1**	**-507.4**	**-36.3**
Paid rent	-236.5	-268.2	-31.7
Paid interest	-244.7	-250.3	-5.6
Received interest	10.1	11.1	1.0

Рисунок 3.4 – Сводная таблица с иерархической структурой

Код R для создания приведенной выше сводной таблицы можно найти в кратком руководстве по регулярной компоновке по адресу http://www.pivottabler.org.uk/articles.

Вычисления могут быть основаны на других вычислениях в сводной таблице. Вычисления могут быть скрыты — например, чтобы скрыть вычисления, которые существуют только для предоставления значений другим вычислениям.

Например, рассмотрим общее количество поездов и процент поездов, прибывающих с опозданием более чем на пять минут, для комбинаций компании-оператора поезда (TOC) и категории поезда представлена на рисунке 3.5.

```
library(pivottabler)
library(dplyr)
library(lubridate)

# derive train delay data
trains <- mutate(bhmtrains,
                 ArrivalDelta=difftime(ActualArrival, GbttArrival, units="mins"),
                 ArrivalDelay=ifelse(ArrivalDelta<0, 0, ArrivalDelta),
                 DelayedByMoreThan5Minutes=ifelse(ArrivalDelay>=5,1,0))

# create the pivot table
pt <- PivotTable$new()
pt$addData(trains)
pt$addRowDataGroups("TOC", totalCaption="All TOCs")
pt$addColumnDataGroups("TrainCategory", totalCaption="All Trains")
pt$defineCalculation(calculationName="TotalTrains", caption="Train Count",
                     summariseExpression="n()")
pt$defineCalculation(calculationName="DelayedTrains", caption="Trains Arr. 5+ Mins
Late",
                     summariseExpression="sum(DelayedByMoreThan5Minutes, na.rm=TRUE)",
                     visible=FALSE)
pt$defineCalculation(calculationName="DelayedPercent", caption="% Late Trains",
```

```
                    type="calculation", basedOn=c("DelayedTrains", "TotalTrains"),
                    format="%.1f %%",

calculationExpression="values$DelayedTrains/values$TotalTrains*100")
pt$renderPivot()
```

	Express Passenger		Ordinary Passenger		All Trains	
	Train Count	5+ mins Late	Train Count	5+ mins Late	Train Count	5+ mins Late
Arriva Trains Wales	3079	13.0 %	830	12.2 %	3909	12.8 %
CrossCountry	22865	15.8 %	63	0.0 %	22928	15.8 %
London Midland	14487	8.8 %	33792	10.5 %	48279	10.0 %
Virgin Trains	8594	10.9 %			8594	10.9 %
All TOCs	49025	12.7 %	34685	10.6 %	83710	11.8 %

Рисунок 3.5 – Сводная таблица, построенная на основе OLAP-куба

Также можно изменить ось (строки или столбцы) и уровень, на котором отображаются расчеты. Подробности см. в виньетке «Расчеты».

Стиль можно задать при создании сводной таблицы. В примере ниже показано задание стиля с использованием функции быстрого поворота и более подробного синтаксиса (рисунок 3.6).

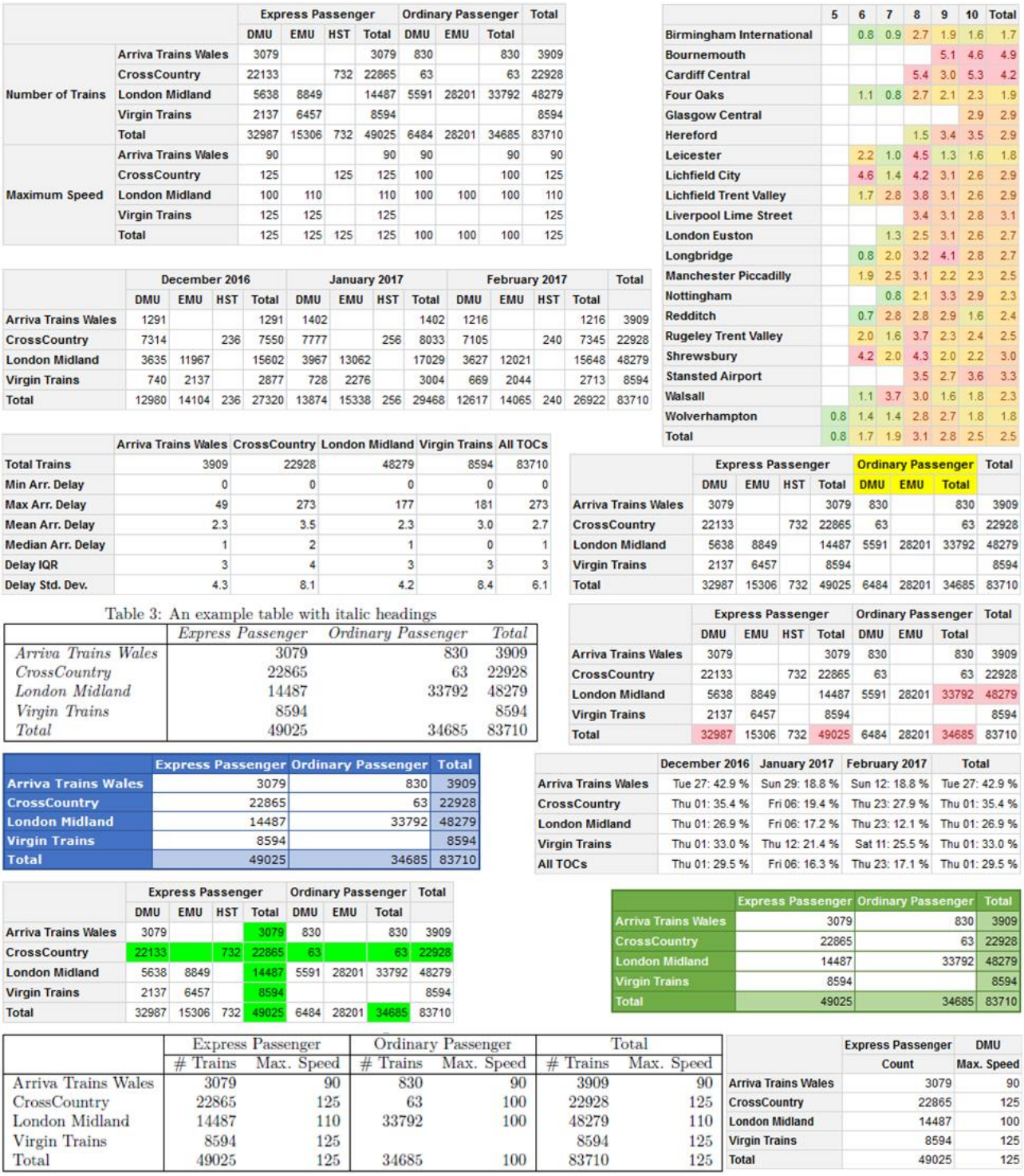
```
qhpvt(bhmtrains, "TOC", "TrainCategory", "n()",
      tableStyle=list("border-color"="maroon"),
      headingStyle=list("color"="cornsilk", "background-color"="maroon",
                        "font-style"="italic", "border-color"="maroon"),
      cellStyle=list("color"="maroon", "background-color"="cornsilk",
                     "border-color"="maroon"),
      totalStyle=list("color"="maroon", "background-color"="cornsilk",
                      "border-color"="maroon", "font-weight"="bold"))

pt <- PivotTable$new(tableStyle=list("border-color"="maroon"),
                     headingStyle=list("color"="cornsilk", "background-color"="maroon",
                                       "font-style"="italic", "border-color"="maroon"),
                     cellStyle=list("color"="maroon", "background-color"="cornsilk",
                                    "border-color"="maroon"),
                     totalStyle=list("color"="maroon", "background-color"="cornsilk",
                                     "border-color"="maroon", "font-weight"="bold"))
pt$addData(bhmtrains)
pt$addColumnDataGroups("TrainCategory")
pt$addRowDataGroups("TOC")
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$renderPivot()
```

	Express Passenger	*Ordinary Passenger*	*Total*
Arriva Trains Wales	3079	830	3909
CrossCountry	22865	63	22928
London Midland	14487	33792	48279
Virgin Trains	8594		8594
Total	49025	34685	83710

Рисунок 3.6 – Сложное форматирование сводной таблицы

На рисунке 3.7 приведены несколько примеров форматированных сводных таблиц.



		Express Passenger				Ordinary Passenger			Total
		DMU	EMU	HST	Total	DMU	EMU	Total	
Number of Trains	Arriva Trains Wales	3079			3079	830		830	3909
	CrossCountry	22133		732	22865	63		63	22928
	London Midland	5638	8849		14487	5591	28201	33792	48279
	Virgin Trains	2137	6457		8594				8594
	Total	32987	15306	732	49025	6484	28201	34685	83710
Maximum Speed	Arriva Trains Wales	90			90	90		90	90
	CrossCountry	125		125	125	100		100	125
	London Midland	100	110		110	100	100	100	110
	Virgin Trains	125	125		125				125
	Total	125	125	125	125	100	100	100	125

	5	6	7	8	9	10	Total
Birmingham International		0.8	0.9	2.7	1.9	1.6	1.7
Bournemouth					5.1	4.6	4.9
Cardiff Central				5.4	3.0	5.3	4.2
Four Oaks		1.1	0.8	2.7	2.1	2.3	1.9
Glasgow Central						2.9	2.9
Hereford				1.5	3.4	3.5	2.9
Leicester		2.2	1.0	4.5	1.3	1.6	1.8
Lichfield City		4.6	1.4	4.2	3.1	2.6	2.9
Lichfield Trent Valley		1.7	2.8	3.8	3.1	2.6	2.9
Liverpool Lime Street				3.4	3.1	2.8	3.1
London Euston			1.3	2.5	3.1	2.6	2.7
Longbridge		0.8	2.0	3.2	4.1	2.8	2.7
Manchester Piccadilly		1.9	2.5	3.1	2.2	2.3	2.5
Nottingham			0.8	2.1	3.3	2.9	2.3
Redditch		0.7	2.8	2.8	2.9	1.6	2.4
Rugeley Trent Valley		2.0	1.6	3.7	2.3	2.4	2.5
Shrewsbury		4.2	2.0	4.3	2.0	2.2	3.0
Stansted Airport				3.5	2.7	3.6	3.3
Walsall		1.1	3.7	3.0	1.6	1.8	2.3
Wolverhampton	0.8	1.4	1.4	2.8	2.7	1.8	1.8
Total	0.8	1.7	1.9	3.1	2.8	2.5	2.5

	December 2016				January 2017				February 2017				Total
	DMU	EMU	HST	Total	DMU	EMU	HST	Total	DMU	EMU	HST	Total	
Arriva Trains Wales	1291			1291	1402			1402	1216			1216	3909
CrossCountry	7314		236	7550	7777		256	8033	7105		240	7345	22928
London Midland	3635	11967		15602	3967	13062		17029	3627	12021		15648	48279
Virgin Trains	740	2137		2877	728	2276		3004	669	2044		2713	8594
Total	12980	14104	236	27320	13874	15338	256	29468	12617	14065	240	26922	83710

	Arriva Trains Wales	CrossCountry	London Midland	Virgin Trains	All TOCs
Total Trains	3909	22928	48279	8594	83710
Min Arr. Delay	0	0	0	0	0
Max Arr. Delay	49	273	177	181	273
Mean Arr. Delay	2.3	3.5	2.3	3.0	2.7
Median Arr. Delay	1	2	1	0	1
Delay IQR	3	4	3	3	3
Delay Std. Dev.	4.3	8.1	4.2	8.4	6.1

	Express Passenger				Ordinary Passenger			Total
	DMU	EMU	HST	Total	DMU	EMU	Total	
Arriva Trains Wales	3079			3079	830		830	3909
CrossCountry	22133		732	22865	63		63	22928
London Midland	5638	8849		14487	5591	28201	33792	48279
Virgin Trains	2137	6457		8594				8594
Total	32987	15306	732	49025	6484	28201	34685	83710

Table 3: An example table with italic headings

	Express Passenger	*Ordinary Passenger*	*Total*
Arriva Trains Wales	3079	830	3909
CrossCountry	22865	63	22928
London Midland	14487	33792	48279
Virgin Trains	8594		8594
Total	49025	34685	83710

	Express Passenger				Ordinary Passenger			Total
	DMU	EMU	HST	Total	DMU	EMU	Total	
Arriva Trains Wales	3079			3079	830		830	3909
CrossCountry	22133		732	22865	63		63	22928
London Midland	5638	8849		14487	5591	28201	33792	48279
Virgin Trains	2137	6457		8594				8594
Total	32987	15306	732	49025	6484	28201	34685	83710

	Express Passenger	Ordinary Passenger	Total
Arriva Trains Wales	3079	830	3909
CrossCountry	22865	63	22928
London Midland	14487	33792	48279
Virgin Trains	8594		8594
Total	49025	34685	83710

	December 2016	January 2017	February 2017	Total
Arriva Trains Wales	Tue 27: 42.9 %	Sun 29: 18.8 %	Sun 12: 18.8 %	Tue 27: 42.9 %
CrossCountry	Thu 01: 35.4 %	Fri 06: 19.4 %	Thu 23: 27.9 %	Thu 01: 35.4 %
London Midland	Thu 01: 26.9 %	Fri 06: 17.2 %	Thu 23: 12.1 %	Thu 01: 26.9 %
Virgin Trains	Thu 01: 33.0 %	Thu 12: 21.4 %	Sat 11: 25.5 %	Thu 01: 33.0 %
All TOCs	Thu 01: 29.5 %	Fri 06: 16.3 %	Thu 23: 17.1 %	Thu 01: 29.5 %

	Express Passenger				Ordinary Passenger			Total
	DMU	EMU	HST	Total	DMU	EMU	Total	
Arriva Trains Wales	3079			3079	830		830	3909
CrossCountry	22133		732	22865	63		63	22928
London Midland	5638	8849		14487	5591	28201	33792	48279
Virgin Trains	2137	6457		8594				8594
Total	32987	15306	732	49025	6484	28201	34685	83710

	Express Passenger		Ordinary Passenger		Total	
	# Trains	Max. Speed	# Trains	Max. Speed	# Trains	Max. Speed
Arriva Trains Wales	3079	90	830	90	3909	90
CrossCountry	22865	125	63	100	22928	125
London Midland	14487	110	33792	100	48279	110
Virgin Trains	8594	125			8594	125
Total	49025	125	34685	100	83710	125

	Express Passenger	DMU
	Count	Max. Speed
Arriva Trains Wales	3079	90
CrossCountry	22865	125
London Midland	14487	100
Virgin Trains	8594	125
Total	49025	125

Рисунок 3.7 – Примеры форматирования сводной таблицы

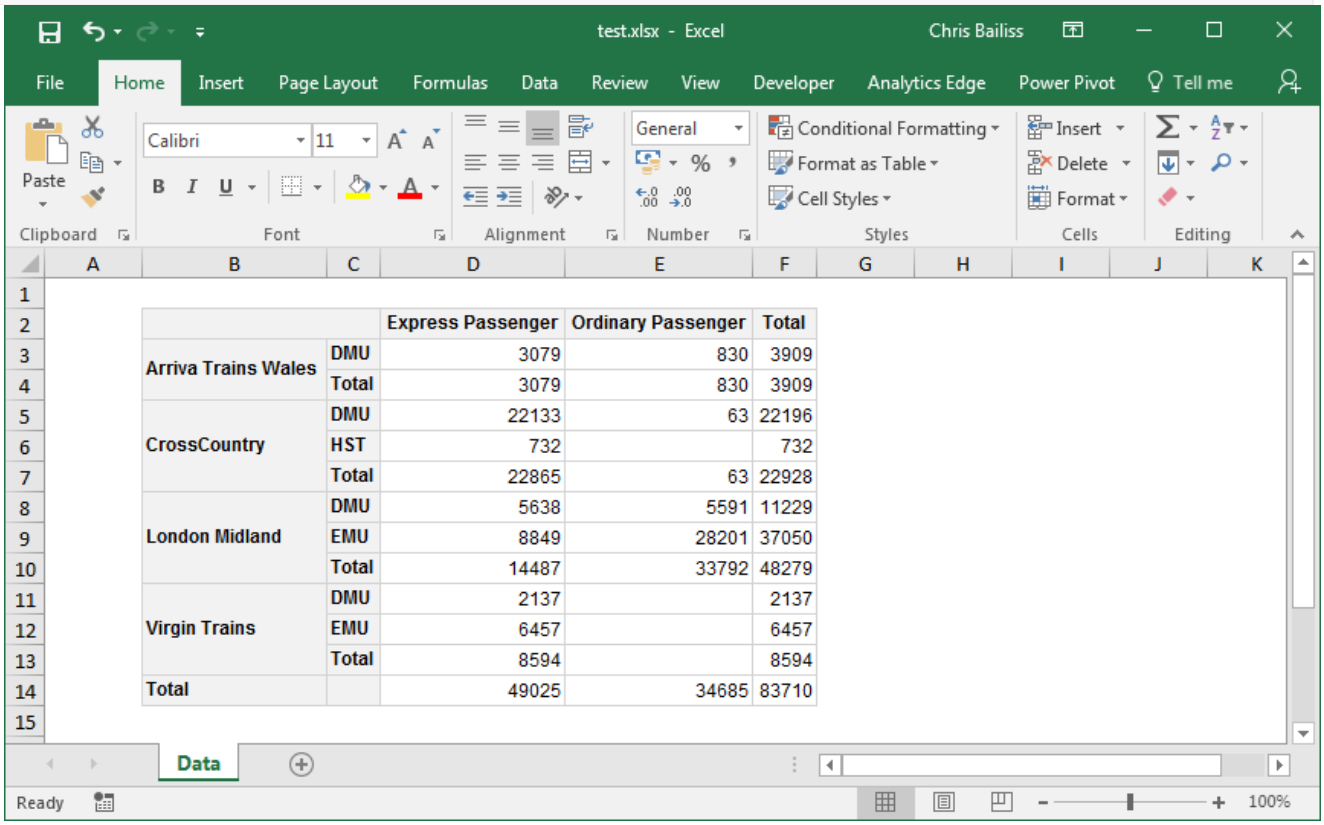
Также можно изменить стиль отдельных ячеек и диапазонов ячеек после создания сводной таблицы. Более подробная информация содержится в разделах «Стиль» и «Поиск и форматирование» на сайте http://www.pivottabler.org.uk/articles.

3.2. Выгрузка данных в формате Excel и Word

Тот же стиль/форматирование, что и для вывода в HTML, применяется и при выводе в Excel, что значительно сокращает объем скрипта, который необходимо написать для создания вывода в Excel (рисунок 3.8).

```
library(pivottabler)
pt <- PivotTable$new()
pt$addData(bhmtrains) # bhmtrains is a data frame with columns TrainCategory, TOC, etc.
pt$addColumnDataGroups("TrainCategory") # e.g. Express Passenger
pt$addRowDataGroups("TOC") # TOC = Train Operating Company e.g. Arriva Trains Wales
pt$addRowDataGroups("PowerType") # D/EMU = Diesel/Electric Multiple Unit, HST=High
Speed Train
pt$defineCalculation(calculationName="TotalTrains", summariseExpression="n()")
pt$evaluatePivot()

library(openxlsx)
wb <- createWorkbook(creator = Sys.getenv("USERNAME"))
addWorksheet(wb, "Data")
pt$writeToExcelWorksheet(wb=wb, wsName="Data",
                         topRowNumber=2, leftMostColumnNumber=2, applyStyles=TRUE)
saveWorkbook(wb, file="C:\\test.xlsx", overwrite = TRUE)
```



		Express Passenger	Ordinary Passenger	Total
Arriva Trains Wales	DMU	3079	830	3909
	Total	3079	830	3909
CrossCountry	DMU	22133	63	22196
	HST	732		732
	Total	22865	63	22928
London Midland	DMU	5638	5591	11229
	EMU	8849	28201	37050
	Total	14487	33792	48279
Virgin Trains	DMU	2137		2137
	EMU	6457		6457
	Total	8594		8594
Total		49025	34685	83710

Рисунок 3.8 – Результаты выгрузки сводной таблицы в формате Excel

На рисунке 3.8 линии сетки были сделаны невидимыми, чтобы сделать стили более заметными (сняв флажок на ленте «Вид»). Размер столбцов также был автоматически изменен, хотя ширину столбцов можно было указать вручную из R. Подробнее см. «Экспорт в Excel» на сайте http://www.pivottabler.org.uk/articles.

3.3. Библиотека ggplot2

Библиотека R **ggplot2** предназначена для визуализации данных. В отличие от многих графических пакетов, ggplot2 использует концептуальную структуру, основанную на грамматике графики (*grammar of graphics*). Это позволяет формировать график из отдельных компонент.

Рассмотрим некоторые компоненты графика и примеры для построения такого графика. Более полную информацию об библиотеке ggplot2 можно найти на сайтах https://ggplot2.tidyverse.org/ и https://cloud.r-project.org/web/packages/ggplot2/index.html.

Последнюю версию пакета ggplot2 можно установить командой:

```
install.packages("ggplot2")
```

Рассмотрим 7 составных частей ggplot2, которые образуют набор инструкций для отображения графиков (рисунок 3.9).

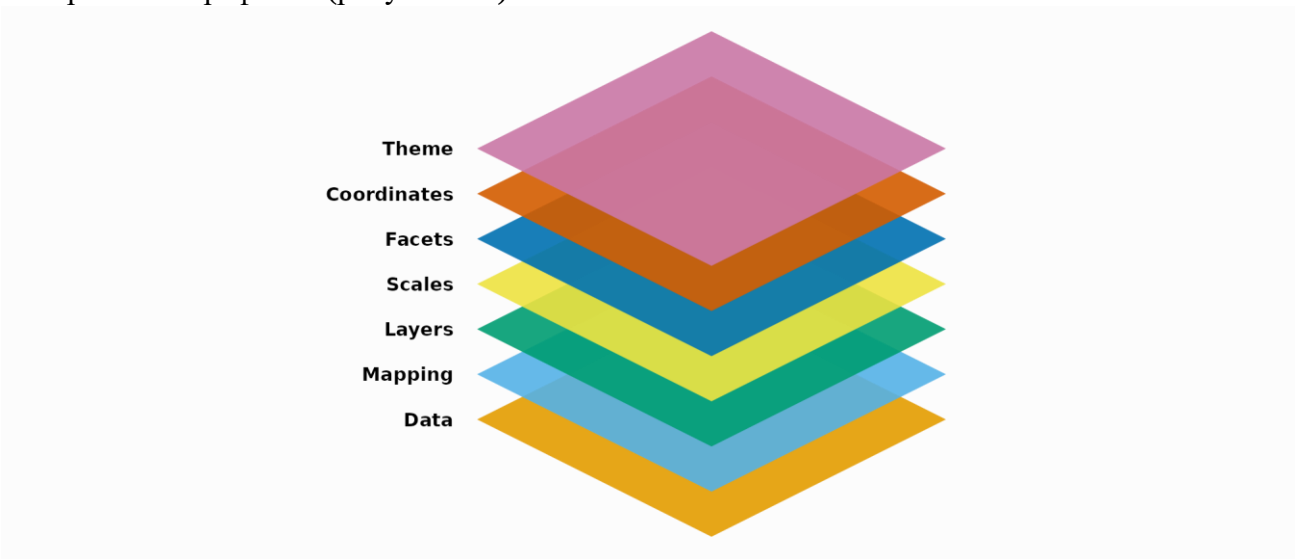


Рисунок 3.9 – Основные компоненты ggplot2 для визуализации данных

Из представленных компонентов ggplot2 необходимы по крайней мере следующие три для создания диаграммы:

- Data (данные),
- Mapping (отображение),
- Layers (слои).

Параметры остальных компонент

- Scales (шкалы),
- Facets (фасеты),
- Coordinates (координаты);
- Theme (темы)

можно оставить по умолчанию.

3.3.1. Данные (Data)

В качестве основы любой графики ggplot2 использует данные для построения графика. Необходимо, чтобы данные были представлены в стандартном формате, то есть имели прямоугольную структуру, где строки — это наблюдения, а столбцы — переменные.

Прежде всего необходимо указать в функцию ggplot() данные, которые в дальнейшем будут использоваться другими компонентами.

Например, если необходимо вывести график по данным mpg, то необходимо выполнить следующую команду:

```
ggplot(data = mpg)
```

3.3.2. Отображение (Mapping)

Отображение графика — это набор инструкций о том, какие данные отображаются на компонентах геометрических объектов. Это «словарь» для перевода данных в графическую систему.

Функция aes() предназначена для отображение графических атрибутов и частей данных. Для отображения на графике столбцов **cty** и **hwy** по координатам x и y, необходимо выполнить следующую команду:

```
ggplot(mpg, mapping = aes(x = cty, y = hwy))
```

3.3.3. Слои (Layers)

Основа любой графики — это слои. Каждый слой состоит из трех частей:

1. Геометрия**,** определяет *способ* отображения данных, например точки, линии или прямоугольники.
2. Статистическое **преобразование**, которое может вычислять новые переменные из данных и влиять на *то, какие* данные отображаются.
3. Выбор **положения**, которая в первую очередь определяет, *где* отображается фрагмент данных.

Слой создаётся с использованием функций geom_*() и stat_*(). Эти функции часто определяют одну из трех частей слоя, в то время как другие две все еще могут быть указаны.

Создадим два слоя для отображения столбцов cty и hwy набора данных mpg в виде точек geom_point() и построим линейную линию тренда geom_smooth() (рисунок 3.10).

```
ggplot(mpg, aes(cty, hwy)) +
  # to create a scatterplot
  geom_point() +
  # to fit and overlay a loess trendline
  geom_smooth(formula = y ~ x, method = "lm")
```

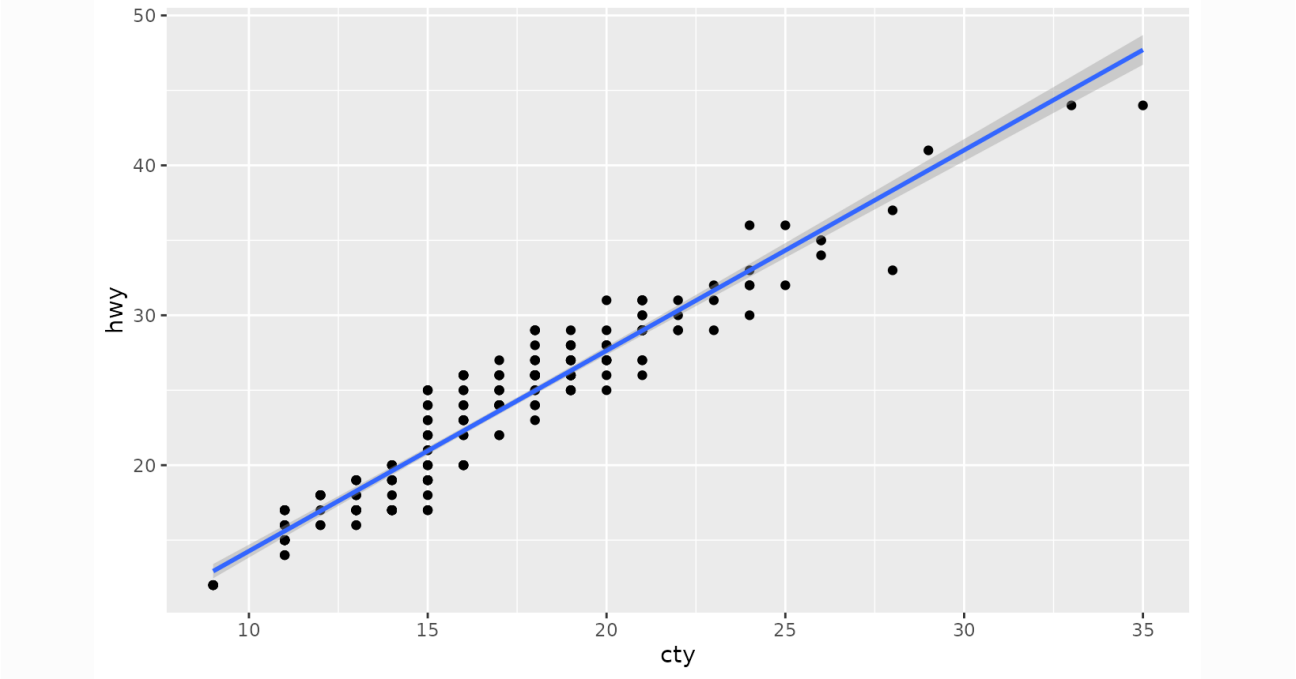


Рисунок 3.10 – Отображение линии тренда на графике ggplot2

3.3.4. Шкалы (Scales)

Шкалы необходимы для того, чтобы понимать что показано на графике. Шкалы обычно образуют пары с эстетическими атрибутами графиков и представлены на графиках направляющими, такими как оси или легенды.

Шкалы отвечают за обновление границ графика, установку разрывов, форматирование меток и, возможно, применение преобразования.

Функций Scales имеют вид:

scale_{aesthetic}_{type}(),

где {aesthetic}— один из параметров отображения графика.

Чтобы сопоставить столбец class в наборе данных mpg с цветовой палитрой viridis необходимо выполнить следующую команду. Результат представлен на рисунке 3.11.

```
ggplot(mpg, aes(cty, hwy, colour = class)) +
  geom_point() +
  scale_colour_viridis_d()
```

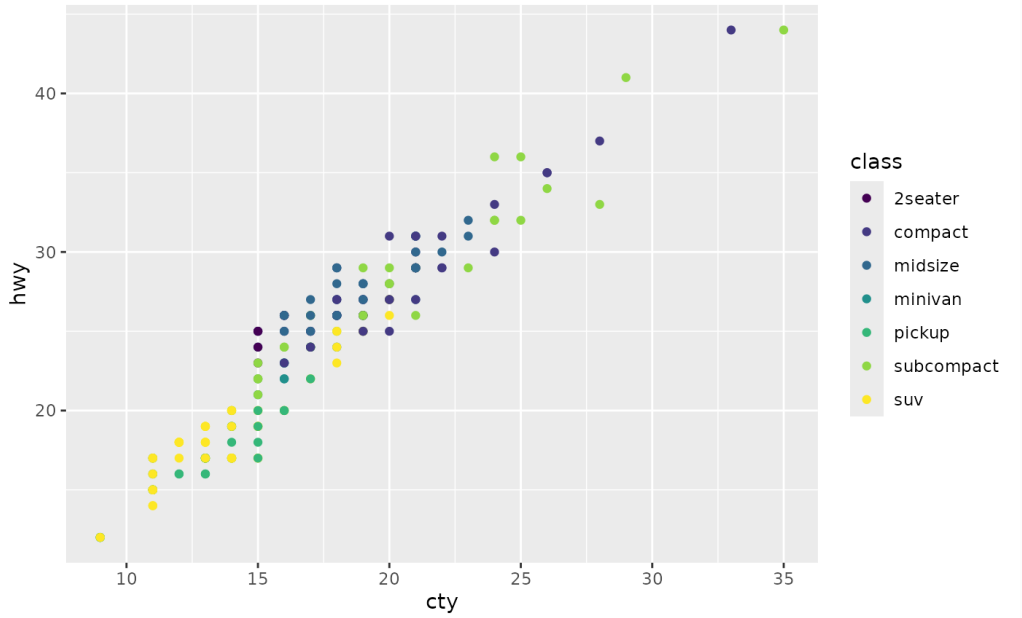


Рисунок 3.11 – Цветовая палитра viridis на графике ggplot2

3.3.5. Фасеты (Facets)

Фасеты можно использовать для раздельного вывода подмножеств данных. Этот инструмент для разделения данных на меньшие диаграммы на основе одной или нескольких переменных для отображения закономерностей или тенденций (или их отсутствия) в подмножествах.

Фасеты имеют собственное отображение, которое может быть представлено в виде формулы. Для построения подмножества набора данных mpg на основе уровней переменных drv и year, можно использовать facet_grid(). Результат представлен на рисунке 3.12.

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point() +
  facet_grid(year ~ drv)
```

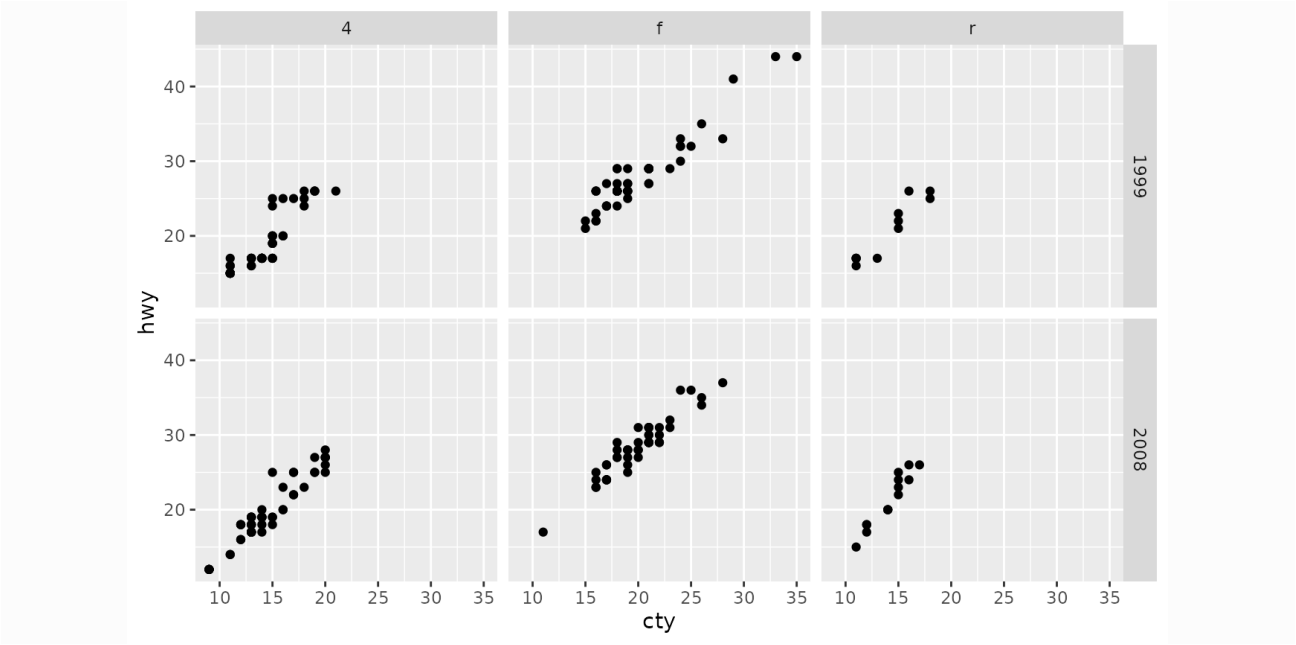


Рисунок 3.12 – Разделение графиков ggplot2 по уровням переменных drv и year

3.3.6. Координаты (Coordinates)

В ggplot2 обычно используются декартовы координаты, но также есть возможность использования система координат обеспечивает отображения в полярных координатах, а также в различных картографических проекциях.

Для отображения графика с фиксированным соотношением сторон, так чтобы по каждой из осей координат имели одинаковую длину, воспользуемся функцией coord_fixed(), которая устанавливает это соотношение автоматически. Результат представлен на рисунке 3.13.

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point() +
  coord_fixed()
```

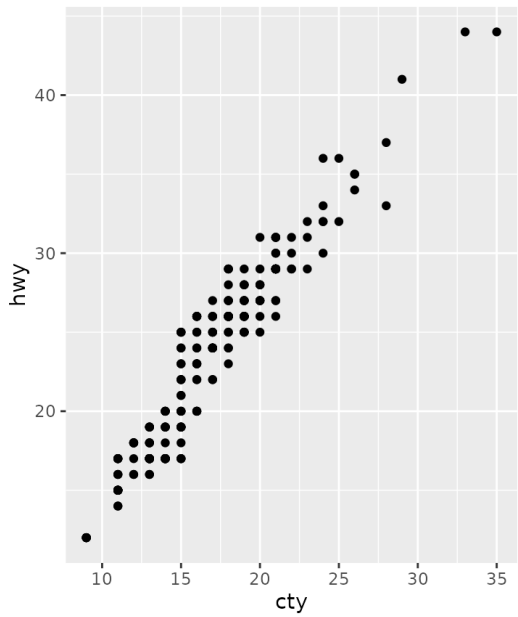


Рисунок 3.13 – Отображение графика ggplot2 с фиксированным соотношением сторон

3.3.7. Тема (Theme)

Темы предназначены для управления практически всеми визуальными элементами графика, которые не определены данными. Темы позволяют изменять внешний вид графика.

Тема используется для настроек, начиная от изменения расположения легенд и заканчивая установкой цвета фона графика. Многие элементы в теме являются иерархическими, в том смысле, что установка внешнего вида линии общей оси одновременно влияет на вид осей x и y.

Чтобы настроить внешний вид графика, можно использовать множество встроенных функций theme_*() и/или детализировать определенные аспекты с помощью функции theme(). Функции element_*() управляют графическими атрибутами компонентов темы.

Пример изменения настроек параметров графика представлен на рисунке 3.14.

```
ggplot(mpg, aes(cty, hwy, colour = class)) +
  geom_point() +
  theme_minimal() +
  theme(
    legend.position = "top",
    axis.line = element_line(linewidth = 0.75),
    axis.line.x.bottom = element_line(colour = "blue")
  )
```

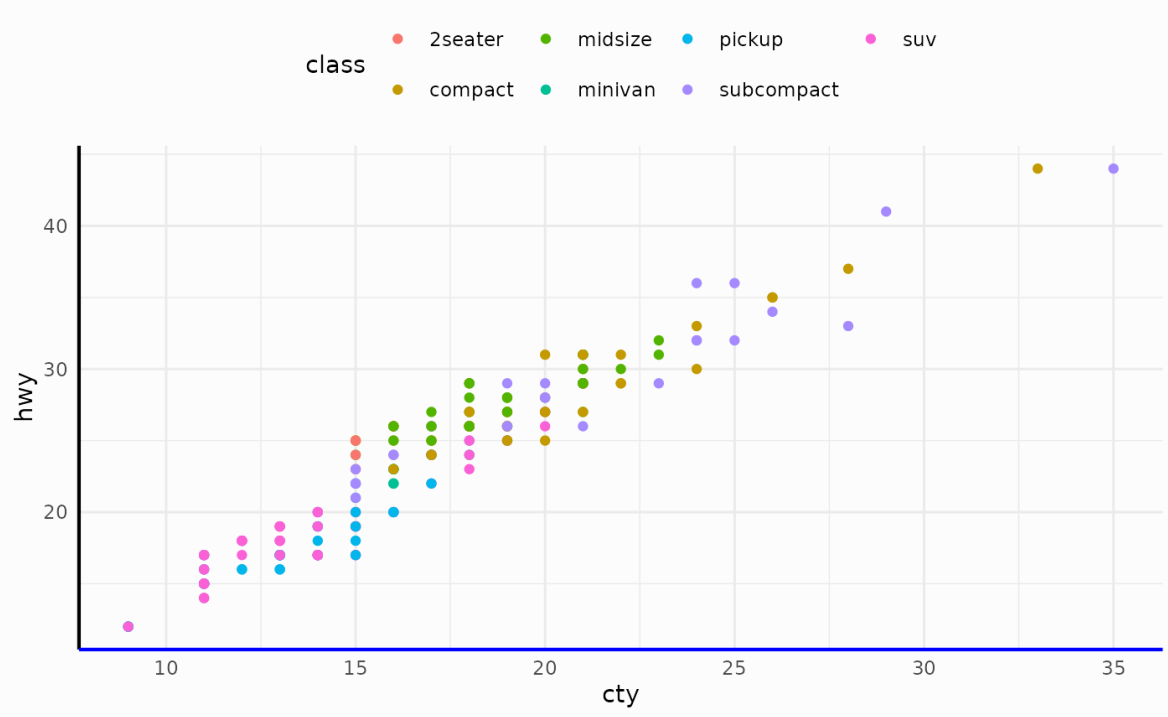


Рисунок 3.14 – Отображение графика ggplot2 с изменёнными настройками

3.3.8. Объединение (Combining)

В конечном счете можно объединить все компоненты, как показано на рисунке 3.15.

```
ggplot(mpg, aes(cty, hwy)) +
  geom_point(mapping = aes(colour = displ)) +
  geom_smooth(formula = y ~ x, method = "lm") +
  scale_colour_viridis_c() +
  facet_grid(year ~ drv) +
  coord_fixed() +
  theme_minimal() +
  theme(panel.grid.minor = element_blank())
```

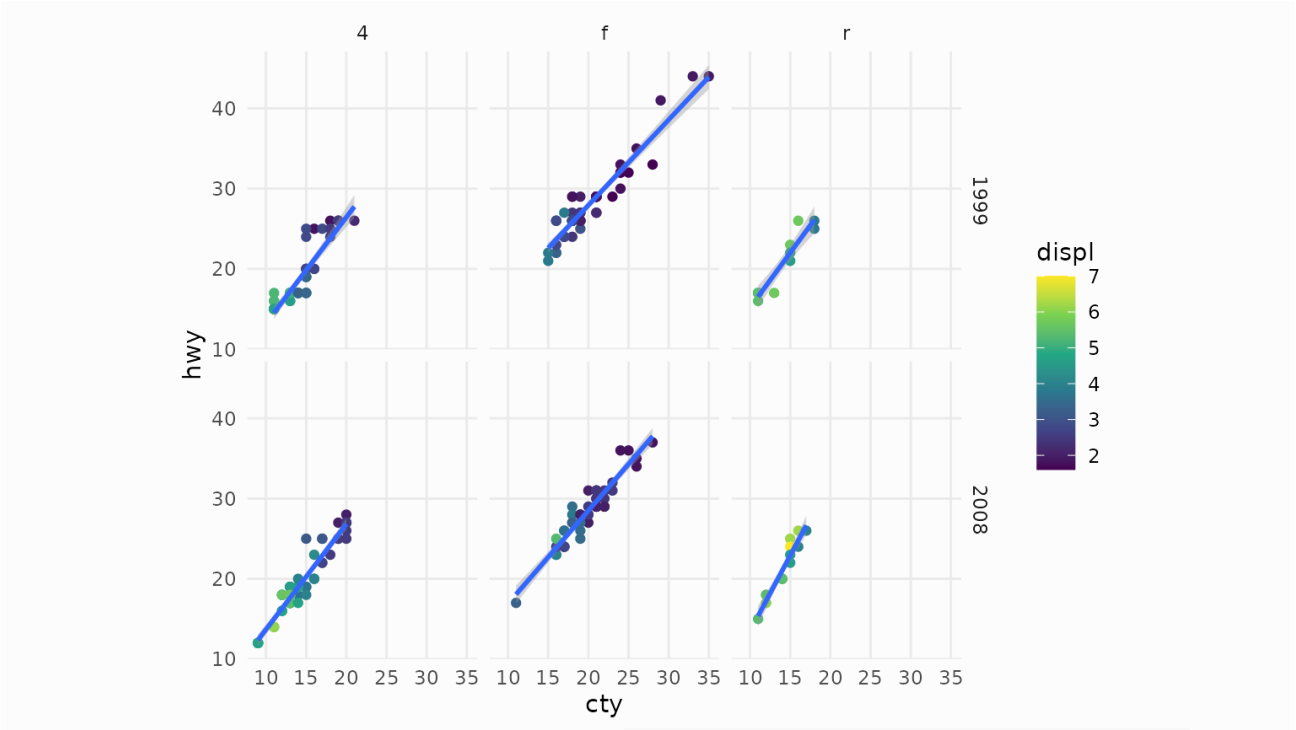


Рисунок 3.15 – Отображение графика ggplot2 после объединения всех настроек

Подробное описание библиотеки ggplot2 содержится в книге ggplot2, а также в списке литературы к данной лабораторной работе.

3.4. Задание на лабораторную работу

1. Подключиться в https://www.kaggle.com/ и скачать данные в формате *.csv в соответствии со своим вариантом (номер варианта соответствует номеру по списку в учебной группе).
2. Отобразить результаты трансформации (аггрегирования) данных в табличном виде в RStudio с использованием библиотеки **pivottabler**.
3. Провести форматирование результатов обработки данных с использованием различных стилей и выгрузить данные в табличном виде в Excel и Word с использованием библиотеки **pivottabler.**
4. Отобразить результаты анализа данных в виде графиков с использованием библиотеки **ggplot2**.
5. Подготовить отчёт по выполненной работе в формате Word/PDF.

3.5. Варианты заданий

1. Latest Global Temperatures (https://www.kaggle.com/datasets/csafrit2/latest-global-temperatures)
2. Finance companies in India (https://www.kaggle.com/datasets/anuragbantu/finance-companies-in-india)
3. Black Friday Sales EDA(https://www.kaggle.com/datasets/pranavuikey/black-friday-sales-eda)
4. Football Transfers from 1992/93 to 2021/22 seasons(https://www.kaggle.com/datasets/cbhavik/football-transfers-from-199293-to-202122-seasons)
5. World, Region, Country GDP/GDP per capita (https://www.kaggle.com/datasets/tmishinev/world-country-gdp-19602021)

6. Air Pollution Level (https://www.kaggle.com/datasets/totoro29/air-pollution-level)
7. Stack Overflow Developer Survey 2011-2022 (https://www.kaggle.com/datasets/yasirabdaali/stack-overflow-developer-survey-20112022)
8. Financing Healthcare (https://www.kaggle.com/datasets/programmerrdai/financing-healthcare)
9. Law School Admissions Bar Passage (https://www.kaggle.com/datasets/danofer/law-school-admissions-bar-passage)
10. Vending Machine Sales (https://www.kaggle.com/datasets/awesomeasingh/vending-machine-sales)
11. Global Companies dataset (https://www.kaggle.com/datasets/narayan63/global-companies-dataset)
12. Smoke Detection Dataset (https://www.kaggle.com/datasets/deepcontractor/smoke-detection-dataset)
13. Netflix Data: Cleaning, Analysis and Visualization (https://www.kaggle.com/datasets/ariyoomotade/netflix-data-cleaning-analysis-and-visualization)
14. World Population by Countries Dataset (1960-2021) (https://www.kaggle.com/datasets/kaggleashwin/population-dataset)
15. Online Retails Sale Dataset (https://www.kaggle.com/datasets/rohitmahulkar/online-retails-sale-dataset)
16. Spotify unpopular songs (https://www.kaggle.com/datasets/estienneggx/spotify-unpopular-songs)
17. Chocolate Bar Ratings (https://www.kaggle.com/datasets/evangower/chocolate-bar-ratings)
18. Airbnb Open Data (https://www.kaggle.com/datasets/arianazmoudeh/airbnbopendata)
19. IMDB Movies (https://www.kaggle.com/datasets/totoro29/imdb-movies)
20. Youtube Statistics (https://www.kaggle.com/datasets/advaypatil/youtube-statistics)

Список литературы

1. *Уикем Х., Гроулмунд Г.* Язык R в задачах науки о данных импорт, подготовка, обработка, визуализация и моделирование данных. – СПб.: ООО «Диалектика», 2018. – 592 с.
2. *Кабаков Р.И.* R в действии. Анализ и визуализация данных на языке R. – М.: ДМК Пресс, 2014. – 588 с.
3. *Bailiss C.* Pivottabler: Создание сводных таблиц – URL: http://www.pivottabler.org.uk/
4. dbplyr: A 'dplyr' Back End for Databases– URL: https://CRAN.R-project.org/package=dbplyr
5. *Lowndes J., Horst A.* R for Excel Users – URL: https://rstudio-conf-2020.github.io/r-for-excel/
6. *Wickham H.* ggplot2: Elegant Graphics for Data Analysis. – New York: Springer-Verlag, 2016. – URL: https://ggplot2.tidyverse.org